Transactions and concurrency - lec 11

ACID

A atomicity: all or nothing. handled by the recovery manager (undo log).
C consistency: the db goes from one valid state to another valid state.
I isolation: concurrent txns should not see each others partial work.
D durability: once committed, stays committed even after a crash (WAL = write ahead log).

Schedules

serial schedule = txns run one after another. serializable schedule = equivalent to some serial one.
a conflict is two ops on the same item, from different txns, where at least one is a write.
conflict serializable iff the precedence graph has no cycle (draw an edge Ti -> Tj if Ti's conflicting op comes first).

What goes wrong without isolation

dirty read: reading data written by a txn that has not committed yet.
non-repeatable read: same row read twice, different values because another txn committed in between.
phantom read: re-running a range query returns new rows.
lost update: two txns overwrite each others change.

Isolation levels (SQL standard)

read uncommitted - allows dirty reads.
read committed - no dirty reads (default in postgres).
repeatable read - no dirty or non-repeatable reads (default in mysql innodb).
serializable - no phantoms either. strictest and slowest.

Locking

shared lock (S) for reads, exclusive lock (X) for writes. S is compatible w/ S only.
2PL = two phase locking: growing phase (only acquire locks) then shrinking phase (only release). guarantees conflict serializability but does NOT prevent deadlock.
strict 2PL: hold all X locks until commit, this avoids cascading aborts.
deadlock = a cycle in the wait-for graph. detect it and abort one victim, or prevent it with timestamps.
wait-die: older txn waits, a younger one that asks for a lock held by an older one is aborted (dies).
wound-wait: older txn wounds (aborts) the younger lock holder, a younger one waits for the older.

Normalization (lec 8)

goal = remove redundancy and update/insert/delete anomalies.
1NF: every attribute atomic, no repeating groups.
2NF: 1NF and no partial dependency (a non-key attribute depending on only part of a composite key).
3NF: 2NF and no transitive dependency (non-key attribute depending on another non-key attribute).

BCNF: for every non-trivial FD X -> Y, X must be a superkey. stricter than 3NF. decomposition should be lossless-join and ideally dependency preserving. BCNF cant always preserve dependencies, 3NF always can.
candidate key = minimal set of attributes whose closure is the whole relation. closure of X = all attributes reachable using the FDs.

Indexing and B+ trees

an index speeds up lookups at the cost of extra space and slower writes.
primary index is on the sorted key of the file, secondary index is on a non-ordering field, clustered means the rows are stored in index order.
B+ tree: all data pointers are in the leaves, leaves are linked together so range scans are fast. internal nodes only guide the search.
order of the tree = max children per node. height stays small (log base order of n) so a lookup is just a few disk reads.
hash index is great for equality lookups but useless for ranges.

Recovery

log-based recovery: write ahead logging means the log record must reach disk BEFORE the changed data page does.
undo for uncommitted txns, redo for committed ones. checkpoint = flush dirty pages and write a checkpoint record, so recovery only scans the log from the last checkpoint.
ARIES has 3 phases: analysis, redo, undo. redo repeats history, then undo rolls back the losers.
shadow paging is the no-log alternative: keep a copy of the page table, swap it in at commit.

SQL joins and deadlock in practice

inner join keeps only matching rows. left outer join keeps all rows of the left table and fills NULLs on the right when there is no match.
a deadlock in a database: txn A locks row 1 then wants row 2 while txn B locks row 2 then wants row 1. the dbms detects the cycle in the wait-for graph and aborts one txn, which the app should retry.
tip from lab: always lock rows in the same order in every txn, that alone avoids most deadlocks.